

ServicePilot – detaillierter Demo-Guide

Stand: 30. August 2026

Dieser Guide ist für eine 20- bis 30-minütige Live-Demonstration vor Fachbereich, IT, Architektur und Management geschrieben. Er enthält:

- die technische Vorbereitung;
- einen sicheren, wiederholbaren Ablauf;
- konkrete Kommandos;
- Eingaben für den Conversational-Agents-Simulator;
- erwartete Ergebnisse;
- einen ausformulierten Sprechertext;
- Alternativen und Recovery-Schritte, falls ein Cloud-Dienst verzögert reagiert.

Die Demo verwendet ausschließlich fiktive Beispieldaten. Keine realen Kundendaten eingeben.

1. Zielbild der Vorführung

Am Ende sollen die Zuhörer vier Eigenschaften verstanden haben:

1. Das Sprachmodell versteht freie Sprache und delegiert an kleine spezialisierte Playbooks.
2. Geschäftsdaten und Transaktionen stammen nie aus dem Sprachmodell, sondern aus einem deterministischen FastAPI-Backend.
3. Technische Antworten werden aus freigegebenen Handbüchern abgerufen und mit Quelle beziehungsweise Abschnitt belegt.
4. Kritische Änderungen benötigen eine exakte Bestätigung und werden erst nach dem kanonischen Backend-Ergebnis als erfolgreich gemeldet.

Der zentrale Satz für die Präsentation lautet:

Wir verwenden generative KI für Sprache, Routing und flexible Dialoge – und deterministischen Code für Identität, Berechtigungen, Verfügbarkeit und Transaktionen.

2. Aktuelle Demo-Umgebung

Ressource	Aktueller Wert
GCP-Projekt	servicepilot-development
Projektnummer	440794212320
Region	europe-west3 (Frankfurt)
Cloud-Run-Service	servicepilot-api
Aktive Revision	servicepilot-api-00002-vwh

Ressource	Aktueller Wert
Runtime-Service-Account	servicepilot-runtime@servicepilot-development.iam.gserviceaccount.com
Artifact Registry	europe-west3-docker.pkg.dev/servicepilot-development/servicepilot
Firestore	(default), Native Mode, europe-west3
Agent-ID	cd9cb6d6-8efa-4cdc-b3af-165a1a0508bf
Agent-Sprache	Deutsch (de)
Agent-Zeitzone	Europe/Berlin
Knowledge Store	servicepilot-knowledge, Standort eu
Knowledge Bucket	gs://servicepilot-development-servicepilot-knowledge

Der Cloud-Run-Service ist nicht öffentlich. Aktuell dürfen der Dialogflow-Service-Agent und der Demo-Benutzer den Service mit `roles/run.invoker` aufrufen.

3. Demo-Datensatz

Die folgenden fiktiven Datensätze sind für die Vorführung vorgesehen:

Typ	Wert	Bedeutung
Kunde	C-10023	Max Mustermann
Gerät	D-1007	HeatPump-X200, Seriennummer SN-4711
Ticket	T-4711	Fehler E37, Status technician_assigned
Termin	A-0815	2. September 2026, 10:00–11:00 Uhr
Aktueller Slot	S-100	durch A-0815 belegt
Freier Slot	S-101	9. September 2026, 14:00–15:00 Uhr
Freier Slot	S-102	10. September 2026, 09:00–10:00 Uhr

Die Demo-Handbücher behandeln unter anderem:

- HeatPump-X200;
- HeatPump-X300;
- Fehlercodes;
- Service-FAQ;
- Garantiebedingungen.

E37 ist absichtlich modellabhängig. Ohne Gerätemodell soll der Agent nachfragen, statt eine Bedeutung zu erfinden.

4. Checkliste am Vortag

4.1 Lokale Voraussetzungen

- Docker Desktop läuft.
- Docker Compose v2 ist verfügbar.
- Google Cloud CLI ist installiert.
- Der verwendete Google-Account hat Zugriff auf das Projekt.
- Der Account besitzt für den direkten Health-Check `roles/run.invoker`.
- Der Conversational-Agents-Simulator lässt sich öffnen.
- Browser-Popups und Google-Anmeldung funktionieren.
- Bildschirmfreigabe zeigt keine Passwörter, Tokens oder private Browser-Tabs.

Prüfen:

```
docker version
docker compose version
gcloud version
gcloud auth list
```

4.2 Repository und Tests

```
Set-Location C:\Users\ralf\OneDrive\Dokumente\git\servicepilot
git status --short
docker compose build
docker compose run --rm api pytest
```

Erwartung zum aktuellen Stand:

```
154 passed, 1 warning
```

Die Warnung betrifft die bekannte Starlette-TestClient-Deprecation und ist kein Fehler der Geschäftslogik.

4.3 Cloud-Zugriff und Zielprojekt

```
$ProjectId = 'servicepilot-development'
$Region = 'europe-west3'
$ServiceName = 'servicepilot-api'
$AgentId = 'cd9cb6d6-8efa-4cdc-b3af-165a1a0508bf'

gcloud config set project $ProjectId
gcloud auth list --filter=status:ACTIVE
gcloud run services describe $ServiceName `
  --project=$ProjectId `
```

```
--region=$Region `
--format='table(metadata.name,status.latestReadyRevisionName,status.url)'
```

4.4 Agent-Ressourcen kontrollieren

Im Conversational-Agents-UI prüfen:

- Agent `ServicePilot`;
- `DefaultService` als Start-Playbook;
- `KnowledgeSupport`;
- `ServiceTicket`;
- `AppointmentManagement`;
- `ComplaintManagement`;
- Tool `ServicePilotBackend`;
- Tool `ServicePilotKnowledge`;
- Flow `AppointmentReschedule`.

Die UI ist unter [Conversational Agents](#) erreichbar. Projekt `servicepilot-development`, Region `europa-west3` auswählen.

4.5 Knowledge Retrieval testen

```
./scripts/test-knowledge-rag.ps1 -ProjectId $ProjectId
```

Erwartet werden:

- `HeatPump-X200-Handbuch` als bester Treffer für `E37/X200`;
- ein Snippet mit „Volumenstrom“;
- `X200-` und `X300-Handbuch` bei einer modellunspezifischen `E37-Suche`.

4.6 Demo-Daten optional zurücksetzen

Der sichere Standardablauf weiter unten verändert keinen Termin. Ein Reset ist dann nicht nötig. Wenn die optionale Write-Demo gezeigt werden soll, vorher den Seed erneut einspielen.

Mit lokalem ADC-Dokument:

```
gcloud auth application-default login
gcloud auth application-default set-quota-project $ProjectId

$env:SERVICEPILOT_FIRESTORE_PROJECT = $ProjectId
$env:SERVICEPILOT_FIRESTORE_DATABASE = '(default)'
$env:SERVICEPILOT_GOOGLE_CREDENTIALS_FILE = `
    "$env:APPDATA\gcloud\application_default_credentials.json"

docker compose -f compose.yaml -f compose.firestore.yaml run --rm `
    api python -m app.seed --confirm
```

Erwartung:

```
Firestore seed completed: customers=5, devices=8, tickets=10,
appointments=8, appointment_slots=11, error_codes=4
```

Der Seed ist absichtlich explizit und läuft nie automatisch beim API-Start.

5. Checkliste 15 Minuten vor der Demo

5.1 Cloud Run vorwärmen

Mit `min-instances=0` kann die erste Anfrage einen Cold Start auslösen. Deshalb vor Beginn einmal authentifiziert aufrufen:

```
$ServiceUrl = gcloud run services describe $ServiceName `
    --project=$ProjectId `
    --region=$Region `
    --format='value(status.url)'
$IdentityToken = gcloud auth print-identity-token
$CloudHeaders = @{ Authorization = "Bearer $IdentityToken" }

Invoke-RestMethod "$ServiceUrl/health" -Headers $CloudHeaders
```

Erwartete Kernaussage:

```
{
  "status": "ok",
  "environment": "production",
  "persistence_backend": "firestore"
}
```

Token nie auf dem Bildschirm ausgeben. Die Variable kann nach der Demo entfernt werden:

Remove-Variable IdentityToken,CloudHeaders **-ErrorAction** SilentlyContinue

5.2 Simulator mit frischer Sitzung öffnen

Für jeden Hauptfall eine neue Sitzung verwenden. Das verhindert, dass ein vorheriger Kontext die nächste Szene beeinflusst. Beim Topic-Switch-Fall dagegen bewusst dieselbe Sitzung behalten.

5.3 Backup-Tabs vorbereiten

Öffnen:

1. Conversational-Agents-Simulator;
2. Cloud Run → **servicepilot-api** → Logs;
3. Firestore → Daten → Collections;
4. Artifact Registry → **servicepilot**;
5. lokale Datei **docs/ARCHITECTURE.md** oder dieses Dokument.

6. Empfohlener Ablauf – 25 Minuten

Szene 1 – Architektur und Sicherheitsprinzip (2 Minuten)

Auf dem Bildschirm

Zeige die Architekturübersicht aus **docs/ARCHITECTURE.md** oder folgende Kurzform:

```

Kunde / Voice
  |
  v
Dialogflow CX: DefaultService -> Spezial-Playbook
  |                               |
  |                               +--> ServicePilotKnowledge -> Agent Search
  |
  v
ServicePilotBackend -> privates Cloud Run -> FastAPI -> Firestore
                        |
                        +--> deterministischer Appointment Flow

```

Sprechertext

ServicePilot ist kein einzelner Prompt und kein autonomes Sprachmodell mit Datenbankzugriff. Der Default-Agent erkennt das Ziel und delegiert an kleine, begrenzte Playbooks. Technische Antworten gehen über einen kuratierten Knowledge Store. Geschäftsdaten gehen über eng definierte OpenAPI-Operationen an ein privates Backend. Besonders wichtig: Das Modell darf keine Termine, Ticketzustände oder Transaktionsergebnisse erfinden. Eine Änderung wird erst nach expliziter Bestätigung, deterministischer Validierung und dem kanonischen Backend-Ergebnis als erfolgreich bezeichnet.

Szene 2 – RAG und kontrollierte Unsicherheit (4 Minuten)

Eingabe 1

Meine Wärmepumpe zeigt Fehler E37. Was soll ich tun?

Erwartetes Verhalten

- Routing zu KnowledgeSupport;
- Rückfrage nach dem Modell, weil E37 modellabhängig ist;
- keine erfundene Bedeutung.

Eingabe 2

Es ist eine HeatPump-X200.

Erwartetes Verhalten

- Aufruf von `ServicePilotKnowledge`;
- Antwort aus dem HeatPump-X200-Handbuch;
- Dokumenttitel und Abschnitt beziehungsweise URI;
- sichere Erstmaßnahmen, keine freie Reparaturanweisung.

Sprechertext

Hier sehen wir bewusst keinen sofortigen „smarten“ Rat. E37 hat in unseren Unterlagen je nach Modell eine andere Bedeutung. Der Agent fragt deshalb nach dem fehlenden Kontext. Erst danach sucht er in den freigegebenen Dokumenten. Die Antwort bleibt nachvollziehbar, weil sie Dokument und Abschnitt nennt. Bei keinem oder widersprüchlichem Treffer benennt der Agent die Wissenslücke, statt sie mit Modellwissen zu füllen.

Optional Negativtest

Neue Sitzung:

Meine HeatPump-X200 zeigt Fehler Z99. Wie repariere ich das?

Erwartung: kein belastbarer Treffer, keine erfundene Reparatur, Angebot eines Servicefalls oder einer menschlichen Übergabe.

Szene 3 – Kanonischer Ticketstatus (3 Minuten)

Eingabe

Wie ist der Stand von Ticket T-4711?

Erwartetes Verhalten

- sofortiges Routing zu ServiceTicket;
- `get_ticket` mit `T-4711`;
- Status aus Firestore, sinngemäß „Techniker zugewiesen“;
- keine Statusbehauptung vor der Tool-Antwort.

Sprechertext

Die Ticketnummer wird nicht als Wahrheit aus der Benutzereingabe übernommen. Das Backend sucht den kanonischen Datensatz. Erst das erfolgreiche Ergebnis erlaubt dem Agenten, den Status zu nennen. Ein unbekanntes Ticket liefert einen strukturierten `TICKET_NOT_FOUND`-Fehler und niemals einen geratenen Status.

Negativtest

Wie ist der Stand von Ticket T-404?

Erwartung: einmalige Bitte zur Prüfung der Kennung, kein Ticketstatus.

Szene 4 – Kombinierte Anfrage und Routing (3 Minuten)

Neue Sitzung:

Meine Anlage zeigt E37 und ich möchte wissen, ob der Technikertermin A-0815 noch steht.

Erwartetes Verhalten

- AppointmentManagement hat Vorrang, weil das explizit gewünschte Ergebnis die Terminprüfung ist;
- E37 bleibt als zusätzlicher Kontext erhalten;
- Termin wird aus dem Backend gelesen.

Sprechertext

Echte Kunden formulieren selten nur eine sauber isolierte Absicht. Der DefaultService priorisiert hier das gewünschte Ergebnis – die Terminprüfung – und übergibt den technischen Kontext mit. Die Spezialisten bleiben trotzdem klein; der Default-Agent führt selbst keine Geschäftstransaktion aus.

Szene 5 – Deterministische Terminänderung, sicher abgebrochen (5 Minuten)

Die Standardvorführung zeigt den vollständigen Bestätigungsmechanismus, führt aber keinen Write aus. Dadurch ist sie beliebig wiederholbar.

Eingabe 1

Ich möchte Termin A-0815 auf den freien Slot S-101 verschieben.

Der Agent soll den aktuellen Termin und den freien Slot aus dem Backend lesen und die exakte Änderung zusammenfassen.

Eingabe 2 – absichtlich mehrdeutig

Ja.

Erwartung: keine Änderung; erneute exakte Zusammenfassung oder Reprompt.

Eingabe 3 – Abbruch

Nein, bitte nichts ändern.

Erwartung: **cancelled**, kein Write, Termin bleibt unverändert.

Backend-Nachweis

```
$IdentityToken = gcloud auth print-identity-token
$CloudHeaders = @{ Authorization = "Bearer $IdentityToken" }
Invoke-RestMethod "$ServiceUrl/appointments/A-0815" -Headers $CloudHeaders |
    ConvertTo-Json -Depth 5
```

Der Termin muss weiterhin Slot **S-100** besitzen.

Sprechertext

Für Transaktionen wechseln wir bewusst von generativer Flexibilität zu einem deterministischen CX Flow. Der Flow erhält nur bereits kanonisch gelesene Termin- und Slotdaten. Ein allgemeines „Ja“ reicht nicht, wenn die exakte Aktion nicht eindeutig im aktuellen Bestätigungskontext steht. Erst der dedizierte Confirmation Intent darf genau einen PUT auslösen. Beim Abbruch sehen wir direkt im Backend, dass nichts verändert wurde.

Optionale echte Write-Demo

Nur zeigen, wenn vorher ein Seed-Reset möglich ist. Statt des Abbruchs exakt bestätigen:

Ja, bitte Termin A-0815 genau auf Slot S-101 verschieben.

Danach erneut per API prüfen. Erwartet:

- `appointment_id = A-0815;`
- `slot_id = S-101;`
- `status = scheduled;`
- S-101 nicht mehr verfügbar;
- S-100 wieder verfügbar.

Nach der Demo Seed erneut ausführen.

Szene 6 – Menschliche Übergabe ohne bekannte Identität (3 Minuten)

Neue Sitzung:

Ich möchte jetzt mit einem Mitarbeiter sprechen. Meine Kundennummer habe ich nicht zur Hand.

Erwartetes Verhalten

- Routing zu ComplaintManagement;
- unmittelbarer `create_handover` mit `reason=human_request`;
- neutrale Zusammenfassung;
- Erfolg nur bei `status=queued` und kanonischer `handover_id`;
- kein Versprechen einer sofortigen Verbindung oder Reaktionszeit.

Sprechertext

Ein ausdrücklicher Menschenwunsch wird nicht durch ein Identitätsformular blockiert. Bekannte Daten werden mitgegeben, unbekannte Daten nicht erfunden. Der Backend-Service erzeugt eine idempotente Übergabe. Der Agent darf nur sagen, dass sie in der Warteschlange steht – nicht, dass bereits ein Mensch verbunden ist.

Hinweis: Diese Szene legt einen neuen fiktiven Handover-Datensatz an. Das ist beabsichtigt und beeinflusst die anderen Demo-Szenen nicht.

Szene 7 – Regression und Ausblick Voice (3 Minuten)

Kommandos

```
docker compose run --rm api pytest `
tests/conversation/test_golden_conversations.py
```

Aktuell werden 40 Golden Conversations in 20 Kategorien geprüft.

Optional Voice-Referenzclient mit vorbereiteten 16-kHz-Mono-WAV-Dateien:

```
$AccessToken = gcloud auth print-access-token
docker compose run --rm `
  -e GOOGLE_OAUTH_ACCESS_TOKEN=$AccessToken `
  api python scripts/voice-session.py `
  --project $ProjectId `
  --region $Region `
  --agent-id $AgentId `
  --input artifacts/voice/live-input/turn-01.wav `
  --input artifacts/voice/live-input/turn-02.wav `
  --output-dir artifacts/voice/demo-output
$VoiceExitCode = $LASTEXITCODE
Remove-Variable AccessToken
if ($VoiceExitCode -ne 0) { throw "Voice demo failed: $VoiceExitCode" }
```

Sprechertext

Generative Antworten ändern ihren Wortlaut. Deshalb testen wir nicht jeden Satz Zeichen für Zeichen, sondern semantische Sicherheitsgrenzen: ausgewähltes Playbook, Tool, Write-Anzahl, Bestätigung, Fehlerpfad und Handover. Derselbe Agent kann über Streaming-STT und -TTS angesprochen werden; Geschäftslogik und Transaktionsgrenzen bleiben identisch. Eine echte Telefonie- oder SIP-Anbindung ist ein nachgelagerter Integrationsschritt.

7. Abschlussstatement

ServicePilot zeigt die Architektur eines produktionsorientierten Service- Agenten: flexible natürliche Sprache, kuratierte Wissenssuche und klar begrenzte Playbooks auf der einen Seite; explizite APIs, transaktionale Datenhaltung, IAM, strukturierte Fehler und überprüfbare Writes auf der anderen. Für einen Kundeneinsatz würden wir als Nächstes die reale Kundenidentität, CRM/ERP, Contact Center, Monitoring-SLOs und den formalen Deployment-Prozess integrieren. Die sicherheitskritische Trennung ist bereits in der Architektur angelegt.

8. Vollständiger Neuaufbau der Cloud-Umgebung

Dieser Abschnitt ist nicht Teil der normalen Live-Demo. Er dient zur Reproduzierbarkeit.

8.1 Variablen

```
$ProjectId = 'servicepilot-development'
$Region = 'europe-west3'
$ServiceName = 'servicepilot-api'
$Repository = 'servicepilot'
$AgentId = 'cd9cb6d6-8efa-4cdc-b3af-165a1a0508bf'
```

```
$RuntimeServiceAccount = `
  'servicepilot-runtime@servicepilot-development.iam.gserviceaccount.com'
```

8.2 Benötigte APIs

```
gcloud services enable `
  run.googleapis.com `
  artifactregistry.googleapis.com `
  cloudbuild.googleapis.com `
  firestore.googleapis.com `
  dialogflow.googleapis.com `
  discoveryengine.googleapis.com `
  storage.googleapis.com `
  secretmanager.googleapis.com `
  --project=$ProjectId
```

8.3 Backend previewen und deployen

```
./scripts/deploy-cloud-run.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -ServiceName $ServiceName `
  -RuntimeServiceAccount $RuntimeServiceAccount `
  -ArtifactRepository $Repository `
  -MinInstances 0 `
  -MaxInstances 3 `
  -Concurrency 40 `
  -MemoryMi 512 `
  -Ingress all `
  -WhatIf

./scripts/deploy-cloud-run.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -ServiceName $ServiceName `
  -RuntimeServiceAccount $RuntimeServiceAccount `
  -ArtifactRepository $Repository `
  -MinInstances 0 `
  -MaxInstances 3 `
  -Concurrency 40 `
  -MemoryMi 512 `
  -Ingress all
```

8.4 Firestore seeden

Den Befehl aus Abschnitt 4.6 ausführen.

8.5 Agent-Ressourcen in richtiger Reihenfolge deployen

Backend-Tool und IAM:

```
./scripts/deploy-conversational-tools.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -AgentId $AgentId `
  -CloudRunServiceName $ServiceName
```

Knowledge Store und Knowledge Tool:

```
./scripts/deploy-knowledge-rag.ps1 `
  -ProjectId $ProjectId `
  -AgentRegion $Region `
  -AgentId $AgentId
```

Auf Abschluss der asynchronen Dokumentindizierung warten, danach testen:

```
./scripts/test-knowledge-rag.ps1 -ProjectId $ProjectId
```

Deterministischen Termin-Flow deployen:

```
./scripts/deploy-appointment-reschedule-flow.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -AgentId $AgentId `
  -CloudRunServiceName $ServiceName
```

Playbooks und Beispiele deployen:

```
./scripts/deploy-conversational-agent.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -AgentId $AgentId
```

Voice-Profil anwenden:

```
./scripts/deploy-voice-agent.ps1 `
  -ProjectId $ProjectId `
  -Region $Region `
  -AgentId $AgentId
```

9. Recovery-Matrix während der Vorführung

Problem	Sofortmaßnahme	Aussage an das Publikum
Erste Antwort langsam	Health-Check erneut ausführen, neue Sitzung	„Cloud Run skaliert in der kostengünstigen Demo auf null; wir wärmen die Instanz kurz vor.“
Simulator behält alten Kontext	Neue Sitzung starten	„Sitzungen sind absichtlich zustandsbehaftet; für isolierte Tests verwenden wir neue Session-IDs.“
RAG liefert noch keinen Treffer	<code>test-knowledge-rag.ps1</code> ausführen; notfalls Architektur zeigen	„Die Dokumentindizierung ist asynchron. Der Agent erfindet bei fehlendem Treffer nichts.“
Cloud Run 401/403	aktiven Account und <code>roles/run.invoker</code> prüfen; neues ID-Token holen	„Ingress-Erreichbarkeit und IAM-Autorisierung sind getrennte Kontrollen.“
Dialogflow Tool 403	IAM-Bindung des Dialogflow-Service-Agenten prüfen	„Das Backend ist privat und akzeptiert nur explizite Service-Identitäten.“
Dialogflow 429 beim Deployment	Deployment erneut starten; Script verwendet begrenztes Backoff	„Das ist ein Control-Plane-Quota, kein Laufzeitfehler des Agenten.“
Termin bereits verändert	Demo-Seed erneut laden oder nur Abbruchpfad zeigen	„Seed und Demo-Write sind bewusst voneinander getrennt.“
Wortlaut weicht ab	Semantisches Ergebnis prüfen	„Generative Sprache variiert; Tool, Datenquelle, Write und Sicherheitszustand sind die geprüften Konstanten.“
Voice-Datei fehlt	Voice-Szene auslassen oder Simulator-Mikrofon verwenden	„Voice ist ein Kanaladapter; dieselbe Geschäftslogik wurde bereits im Chat demonstriert.“

10. Nachbereitung

```
Remove-Variable IdentityToken,AccessToken,CloudHeaders -ErrorAction
SilentlyContinue
docker compose down
```

Falls die optionale Termin-Write-Demo verwendet wurde, Demo-Seed erneut laden. Prüfen, ob während der Präsentation neue Handover-Datensätze entstanden sind; sie enthalten nur fiktive Demo-Daten.

Weiterführende technische Details stehen in [docs/TECHNICAL_GUIDE.md](#).